



Google Cloud

Blog

Contact sales

Get started for free

 Google Cloud

Blog

[Contact sales](#)

[Get started for free](#)

Threat Intelligence

Using capa Rules for Android Malware Detection

February 7, 2025

Lin Chen



Mobile devices have become the go-to for daily tasks like online banking, healthcare management, and personal photo storage, making them prime targets for malicious actors seeking to exploit valuable information. Bad actors often turn to publishing and distributing malware via apps as a lucrative channel for generating illegal and/or unethical profits.

Android takes a multi-layered approach to combating malware to help keep users safe (more later in the post), but while we continuously strengthen our defenses against malware, threat actors are persistently updating their malware to evade detection. Malware developers used to complete their entire malicious aggression using the common Android app development toolkits in Java, which is easier to detect by reversing the

to obfuscate some of the critical malware behaviors and putting their hopes on obscurity in compiled and symbol-stripped Executable and Linkable Format (ELF) files, which can be more difficult and time-consuming to reveal their true intentions.

To combat these new challenges, Android Security and Privacy Team is partnering with Mandiant FLARE to extend the open-source binary analysis tool [capa](#) to analyze native ARM ELF files targeting Android. Together, we improved existing and developed new capa rules to detect capabilities observed in Android malware, used the capa rule matches to highlight the highly suspicious code in native files, and [prompted Gemini](#) with the highlighted code behaviors for summarization to enhance our review processes for faster decisions.

In this blog post, we will describe how we leverage capa behavior-detection capabilities and state-of-art Gemini summarization by:

- Showcasing a malware sample that used various anti-analysis tricks to evade detections
- Explaining how our existing and new capa rules identify and highlighted those behaviors
- Presenting how Gemini summarizes the highlighted code for security reviews

App Under a Music App Façade

Google Play Store ensures all published apps conform to local laws and regulations. This includes gambling apps, which are prohibited or require licenses in some areas. Developing and distributing illegal gambling apps in such areas can generate significant illicit profits, which sometimes is [associated with organized crimes](#). To bypass Google Play Store's security-screening procedures, some gambling apps disguise themselves with harmless façades like music or casual games. These apps only reveal their gambling portals in certain geographic markets using various anti-analysis tricks. Unfortunately, dynamic analysis, such as emulation and sandbox detonation, relies on specific device configurations, and threat actors keep trying different combinations of settings to evade our detections. It's an ongoing game of cat and mouse!

In response, the Android Security and Privacy Team has evolved static analysis techniques, such as those that evaluate the behavior of a complete program and all its conditional logic. So, let's describe an app that violated Google Play Store rules and show how we can better detect and block other apps like it.

We received reports of a music app opening gambling websites for users in certain geographical areas. It used an interesting trick of

and is loaded at runtime to evade detection.

When we decompiled the app into Java source code, using a tool like [JEB Decompiler](#), we found that the app has a song-playing functionality as shown in "MainActivity" of Figure 1. This looks like benign behavior and is fully within the limits of Google Play Store policies.

However, there was a small region of initialization code that loads an ELF file as soon as the app is initialized when calling the `onCreate` function, as shown in `com.x.y.z` class of Figure 1. To fully understand the behavior of the entire app, we also had to reverse engineer the ELF file, which requires a completely different toolset.

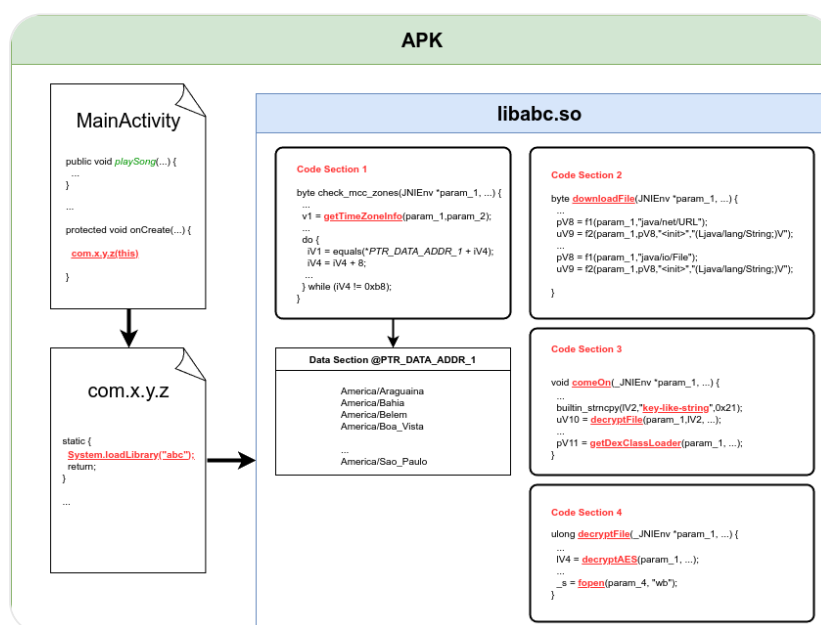


Figure 1: How the app applies anti-analysis techniques

Using a tool like [Ghidra](#), we decompiled the ARM64 ELF file into C source code and found that this app estimates the user's geographic location using timezone information ("Code Section 1" in

regions ("Data Section" in Figure 1).

If the user's location matches a value in the list ("Data Section" in Figure 1), this malware:

1. Downloads an encrypted DEX file from a remote server ("Code Section 2" in Figure 1)
2. Decrypts the downloaded DEX file ("Code Section 3" in Figure 1)
3. Loads the decrypted DEX file into memory ("Code Section 4" in Figure 1)

The loaded DEX file uses further server-side cloaking techniques and finally loads a gambling website (Figure 3) to the app users. Compared to the app icon in Figure 2, it is an obvious mismatch of the app's advertised functionality.



Figure 2: The app icon as published

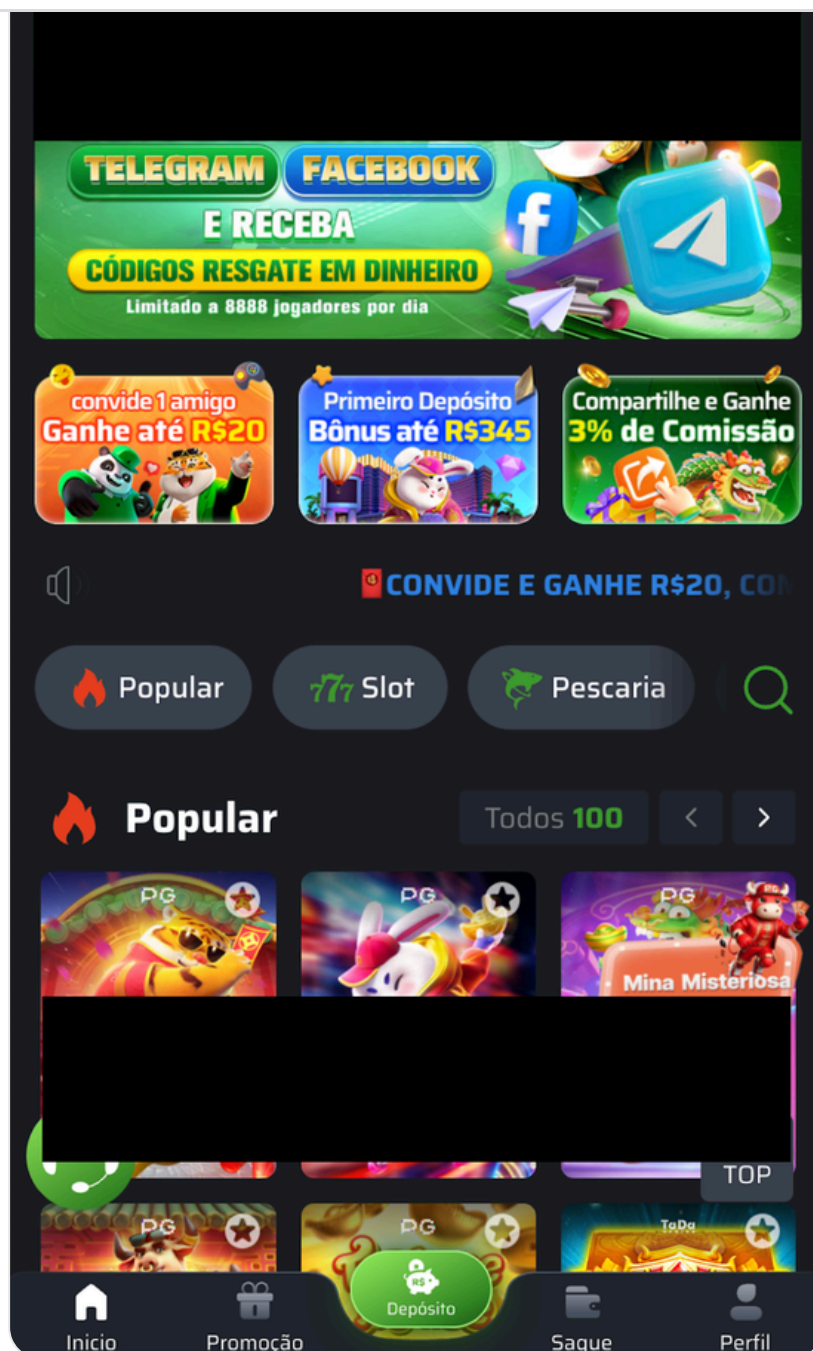


Figure 3: The loaded gambling website in app

While there are many detection technologies, such as [YARA](#), available for identifying malware distributed in ELF files, they are less resilient to app updates or variations introduced by threat actors. Fortunately, the Android Security and Privacy Team has developed new techniques for detecting malicious Android apps by inspecting their native ELF components. For example, in the

(JNI) that interact with the Android runtime. Our detection systems recognized these cross-runtime interactions and reason about their intent. We've enumerated behaviors commonly seen in Android malware, such as making `ptrace` API calls, extracting device information, downloading code from remote servers to local storage, and making various cryptographic operations via JNI, turning them into capa detections we can use to identify and block Google Play Store threats.

Let's now talk a little more about how this works.

Android capa Rules

[capa](#) is a tool that detects capabilities in executable files. You run it against a compiled program, and it tells you what it thinks the program can do. For example, capa might suggest that a file is a backdoor, is capable of installing services, or relies on HTTP to communicate.

Mandiant FLARE extended capa to support [BinExport2](#), an architecture agnostic representation of disassembled programs. This enables capa to match capabilities for additional architectures and file formats, such as those supported by Ghidra and its [BinExport2 plugin](#), with an initial focus on ARM64 ELF files. The Android Security and Privacy Team then created new capa rules focused specifically on detecting capabilities observed in ARM64 ELF files used by various Android malware samples. These proprietary rules alongside [capa's open-source](#)

pipelines.

Referring back to the gambling app in Figure 3, the following Google proprietary rules and open-source capa rules matched the malicious functions performing cloaking techniques for further inspection.

Proprietary rules:

- Make ptrace API calls
- Extract device configuration information via JNI on Android
- Extract timezone via JNI on Android
- Encode or decode data using Base64 via JNI on Android
- Encrypt or decrypt data using Cipher API via JNI on Android

Open-source capa rules:

- [Create or open file](#)
- [Write file on Linux](#)
- [Read file on Linux](#)
- [Check file permission on Linux](#)
- [Create thread](#)
- [Reference Base64 string](#)

Instead of browsing hundreds of thousands lines of obfuscated code, our analysts were able to quickly identify the evidence of the app's wrongdoings using the function addresses matching those rules and enforced on the app.

capa Rule Matches

Safeguarding the Android ecosystem, our Android malware analysis pipelines scan millions of ELF files in-depth every day, each one containing thousands to millions of lines in their decompiled codes. On top of the [fast-evolving Gemini capabilities in malware analysis](#), capa rules are able to select the most interesting code for Gemini summarization, with sharpened focus on a much smaller set of the most suspicious functions.

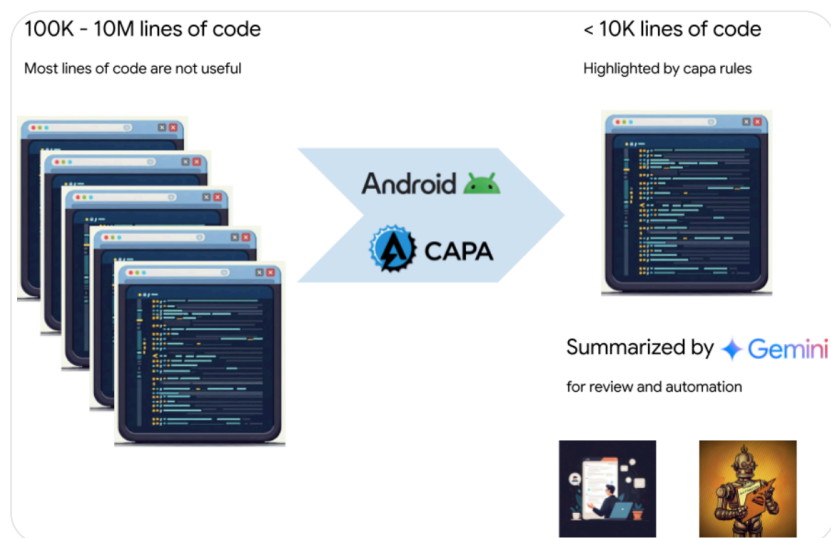


Figure 4: How capa rules and Gemini help our reviewers and automation

We asked Gemini to summarize the functions matched on capa rules from the earlier gambling app with the following prompt:

```
You are a world-class Android  
reverse engineer with malware  
analysis specialities. You are given  
a few functions from an ARM/AArch64
```

you need to analyze these functions to

1. summarize what these functions try to achieve

2. decide whether any of them has code/behavior patterns that would be found in Android malware.

{some more instructions...}

...

These functions are decompiled by Ghidra into pseudo-C format as follows:

[function_1 content]

[function_2 content]

...

In your response, please give your overall verdict assessing the risk level using HIGH / MEDIUM / LOW in the beginning, and explain your rationale for each potential malicious behavior.

Gemini responded with the following suggestions:

VERDICT: HIGH RISK

indicative of malicious intent. The risk level is assessed as HIGH due to the presence of multiple red flags consistent with sophisticated Android malware techniques. The analysis below details the specific malicious behavior patterns observed:

1. Dynamic Code Loading (HIGH): The function `comeOn` demonstrates a pattern of dynamically loading and executing code. The app fetches files (potentially DEX or ELF) using `getCacheFilePath` and `downloadFile`, then attempts to execute them using `getDexClassLoader`. This is a classic technique for delivering malicious payloads after initial installation, allowing the malware to easily update its functionality and evade static analysis. The use of custom decryption (`decryptFile`, `decryptAES`) further enhances the obfuscation and anti-analysis capabilities, indicating a deliberate attempt to conceal the loaded code's true nature.

2. Custom Decryption and Execution (HIGH): The `decryptFile` function performs custom decryption of a file, likely to obfuscate the malicious payload. The decryption utilizes an AES algorithm, which adds a layer of complexity to the reversal process. In combination with the dynamic loading, this points strongly towards a sophisticated malware strategy. The decrypted file is then written to disk (`fopen`, `fwrite`) and executed using a custom class loader.

cloaking. It checks the user's timezone (`getTimeZoneInfo`) and potentially alters behavior if the timezone is not "America/Sao Paulo". More importantly, it appends a timestamp (`"?time=" + current time`) to a string used for downloading further files. This suggests an attempt to evade detection by changing behavior based on the device's time and potentially making analysis more challenging by dynamically creating different network requests.

4. Network Cloaking (MEDIUM):

`thread_function` exhibits additional cloaking behavior, though details are limited. The function interacts with network resources to download additional resources, showing its network-active nature. The data used in network operations are likely obfuscated through string manipulation and URL obfuscation, making identification and analysis difficult. The precise cloaking mechanisms used are not fully clear from the decompiled code, but the presence of this behavior is a significant concern.

5. Obfuscation Techniques (HIGH): The entire codebase uses extensive obfuscation techniques. The code is heavily reliant on complex data structures (deques, hash tables, trees) and intricate control flow. Function and variable names are nonsensical, hindering analysis. The use of multiple layers of indirection, combined with aggressive heap memory management (`operator_new` ,

methods are typical of malware aiming to evade reverse engineering.

6. Anti-Debugging/Anti-Analysis

Techniques (HIGH): The `JNI_OnLoad` function includes a call to

`ptrace(PT_TRACE_ME, 0, 0, 0)`. This is a clear indicator of an attempt to detect debugging or analysis tools. If a debugger is attached, this function's behavior will likely differ; it could crash, stall the execution, or even hide its malicious code.

Conclusion:

The combination of dynamic code loading, custom decryption, sophisticated obfuscation, and anti-debugging techniques creates a high likelihood of malicious activity. The incomplete understanding of the role of several custom functions further aggravates the risk. The app's behavior is heavily contingent on received data and time-sensitive factors, raising serious concerns about its potential for wide-ranging malicious acts. Immediate action should be taken to contain and investigate this application further. A detailed sandbox analysis is critical to identify any network communications, file system manipulation, or other abusive activities.

As seen in the Gemini output, the Android ELF behaviors are explained clearly on the functions matched on capa rules.

perform dynamic code loading, where our analysts can easily inspect the key function calls `getCacheFilePath` and `getDexClassLoader`

- Identify the timezone extraction with the additional URL parameter hint, where our analysts may try to probe the malicious payload quickly and accurately
- Describe more potential suspicious behaviors (e.g. `getDexClassLoader` JNI call, URL obfuscation) for further rule-writing ideas

capa rules in Android together with Gemini summarization shows great potential for further malware detection with more advanced techniques. Our analysts are closely monitoring the malware trends and techniques in the market and writing up-to-date capa rules to catch the bad actors in the wild.

Android's Multi-Layered Security Approach

Android's ever-evolving, multi-layered security approach includes integrating advanced features and working with developers and device implementers to keep the Android platform and ecosystem safe. This includes, but is not limited to:

Android devices with Google Play Services, no matter the download source. This built-in protection, enabled by default, provides crucial security against malware and unwanted software. Google Play Protect scans more than 200 billion apps daily and performs real-time scanning at the code-level on novel apps to combat emerging and hidden threats, like polymorphic malware. In 2024, Google Play Protect's [real-time scanning](#) identified more than [13 million new malicious apps from outside Google Play](#).

- **Google Play and developer protections from malware:** To create a trusted experience for everyone on Google Play, we use our [SAFE principles](#) as a guide, incorporating multi-layered protections that are always evolving to help keep Google Play safe. These protections start with the developers themselves, who play a crucial role in building secure apps. We provide developers with [best-in-class tools](#), [best practices](#), and on-demand training [resources](#) for building safe, high-quality apps. Every app undergoes rigorous [review and testing](#), with only approved apps allowed to appear in the Play Store. Before a user downloads an app from Play, users can explore its user reviews, ratings, and [Data safety section](#) on Google Play to help them make an informed decision.
- **Engagement with the security research community:** Google works closely with the security community on multiple levels, including the [App Defense Alliance](#), to

Group (GNG) to address emerging threats and safeguard Android users worldwide.

Equipped with the fast-evolving Gemini, our analysts are able to spend less time on those sophisticated samples, minimising the exposure for malicious apps and ensuring the safety of Android ecosystems.

Acknowledgement

Special thanks to Willi Ballenthin, Yannis Gasparis, Mike Hunhoff, and Moritz Raabe for their support.

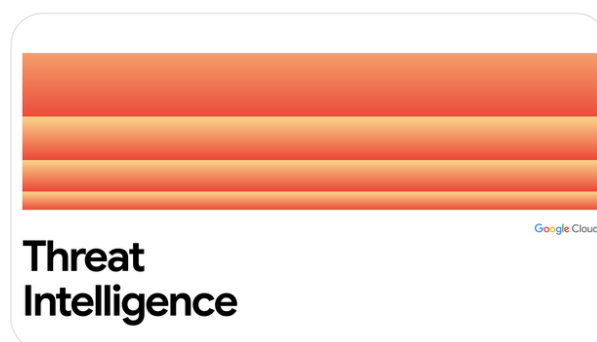
Posted in [Threat Intelligence](#)

Related articles



Threat Intelligence

**Google's Continued Disruption
of Malicious Residential Proxy
Networks**



Threat Intelligence

**The Bear Necessities: A Look at
the Drivers, Dynamics, and
Applications of the Pro-Russia
Influence Ecosystem**



Threat Intelligence

Threat Intelligence

STOCKSTAY Another Day: The Latest Addition to Turla's Intelligence Gathering Apparatus

By Google Threat Intelligence Group • 68-minute read



Threat Intelligence

Threat Intelligence

Zero-Day Exploitation of Vulnerability (CVE-2026-20245) in Cisco Catalyst SD-WAN Manager

By Mandiant • 16-minute read

Follow us



Google Cloud

Google Cloud Products

Privacy

Terms



Help

English